SatsPath Architecture

Overview

SatsPath is a routing and resolution layer that sits above existing Bitcoin payment protocols. It does not replace Lightning, Ark, or on-chain payments — it resolves human-readable identifiers to signed payment profiles and selects the best available rail.

System Diagram

```
Human-readable identifier
  (e.g. rodrigo@satspath.dev)
          |
          v

    Resolver /
    Registry           <  local .satspath/registry.json (prototype)
                       <  BIP-353 / HTTPS / Nostr / optional P2P

           |
           v

    Signed Payment
    Profile

    - alias
    - identity_pubkey
    - methods[]
    - signature

           |
           v

    Route Engine

         |    |    \
         v    v     v

     LN    BTC     Ark

       |       |        |
       v       v        v
  Invoice   PSBT    Ark Intent
```

Crate Structure

```
satspath/
  crates/
     satspath-core/          # Types, crypto, codec, registry
        profile.rs          # PaymentProfile, PaymentMethod, SignedPaymentProfile
        crypto.rs           # secp256k1 keypair, sign, verify, fingerprint
        codec.rs            # encode/decode satspath: URIs
        registry.rs         # local file registry
        errors.rs           # SatsPathError enum
```

```
satspath-router/        # Route selection engine
    router.rs           # select_route() - priority logic
    fees.rs             # mempool.space fee API client
    lightning.rs        # Lightning availability + fee helpers
    onchain.rs          # On-chain fee math + availability
    ark.rs              # ArkClient trait + MockArkClient

satspath-cli/           # User-facing binary
    commands/           # One module per CLI subcommand
```

Data Flow

Payment request lifecycle

```
1. Sender knows: rodrigo@satspath.dev + 21000 sats

2. Encode (optional):
   satspath:v1:<base64url({"version":1,"alias":"rodrigo@satspath.dev","amount_sats":21000,...})>

3. Decode:
   PaymentRequest { alias, amount_sats, memo, ... }

4. Resolve:
   Registry::resolve_alias("rodrigo@satspath.dev") -> SignedPaymentProfile

5. Verify:
   verify_signed_profile(signed) -> bool

6. Route:
   select_route(RouteRequest { alias, amount_sats, signed_profile })
   -> RouteQuote { selected_method, reason, fee, confirmation }

7. Pay (simulated):
   Execute against selected rail (Lightning invoice, on-chain tx, Ark intent)
```

Invite lifecycle (unregistered receiver)

```
1. Sender tries to pay julian@example.com
2. Registry returns AliasNotFound
3. create_invite("julian@example.com", amount_sats) -> Invite
4. Invite contains alias_hash, claim_url, warning
5. Sender shares claim_url with receiver out-of-band
6. Receiver generates their own keys locally and registers
7. Sender retries payment
```

Security Boundaries

- Private keys never leave the user's machine.
- The registry stores only public data (pubkeys, addresses, signatures).
- Signature verification happens on the receiver side before any payment.
- The `.satspath/` directory is git-ignored.

Pluggability

The registry is an abstraction — in production, `Registry::open()` can be swapped for: - BIP-353 DNS TXT record lookup - Nostr NIP-05 / NIP-57 resolution - Lightning Address `.well-known/lnurlp` discovery - A decentralized DHT

The router is similarly pluggable: new payment rails are added by implementing the routing priority logic and returning a `RouteQuote`.